

An Exact Snoop Filter for Seamless Cache Coherence in FPGA-Based Multi-Core RISC-V

Anonymous submission

Abstract—Private data caches allow the soft cores of an FPGA-based multi-core system to hide the access latency of a shared memory, but the caches then have to be kept coherent. A recently proposed cache extension for RISC-V multi-core systems does this in hardware, with a snooping invalidation bus and without any modification to the core. In this paper we look at the traffic on that bus. We reproduce the cache extension at the register-transfer level, instrument its snoopy bus, and count how many invalidation requests actually invalidate a cache line in another core. On the benchmark kernels of the original evaluation the answer is none. The kernels partition their data between the processing elements, so every written line is private to the core that writes it, and yet each write is still arbitrated across the whole system and acknowledged by every other cache. A considerable fraction of each core’s cache busy time is spent waiting for the snoopy bus in this way, and for a streaming kernel the overhead is larger than the benefit of the cache. We propose a snoop filter that keeps a copy of the tag and valid bits of every data cache unit inside the snoopy bus, so that the bus can suppress an invalidation request when no other cache holds the addressed line or is fetching it. Conventional snoop filters sit at the receiving cache and work from approximate or coarse-grained sharing information. Ours is exact and sits at the source, where it becomes part of the arbitration decision itself, because on this bus the dominant cost of an unnecessary invalidation is the global serialization it causes and not the remote tag lookups. A cost model of the bus protocol explains where the saving comes from. The filter is evaluated in terms of execution time, coherence correctness and FPGA resource utilization.

Index Terms—Cache Coherency, Snoop Filter, RISC-V, Multi-Core SoC, FPGA

I. INTRODUCTION

Multi-core systems implemented on FPGAs are increasingly used for embedded and edge computing, where several soft RISC-V cores share a common memory whose access latency is high compared to the clock period of the cores. Private data caches hide this latency. However, as soon as each core caches shared data the copies have to be kept coherent, and the cost of coherence is charged against the very performance the caches were added to provide, partly as hardware for tag and status memories and partly as the time the cores spend waiting for coherence actions that are serialized across the whole system.

Several hardware and software coherence schemes have been proposed for scalable architectures [2]–[4], and the RISC-V ecosystem provides coherent interconnect fabrics for large systems [5]–[7]. Adopting one of these generally means integrating with its coherence and interconnect infrastructure, and it may require changes to the core or to its memory interface. For a designer working with existing, already verified soft cores, that is the development cost worth avoiding, and a seamless cache extension avoids it. Kamaleldin *et al.* [1]

attach a private, write-through Data Cache Unit (DCU) to an unmodified RISC-V core through the core’s ordinary data port and keep the DCUs coherent with a snooping invalidation bus, entirely in hardware.

In this work we reproduce the cache extension of [1] at the register-transfer level in order to find out how much of the coherence traffic it generates is really necessary. We find that between 6 % and 33 % of the time a core spends with an access in its cache goes to waiting for the snoopy bus, and that over four benchmark kernels the system issues 1,048,821 invalidation requests, not one of which invalidates a cache line in another DCU. The kernels partition their arrays between the Processing Elements (PEs), so almost every written line is private to one PE, but the bus has no way of knowing this. We therefore propose an exact snoop filter that answers this question before the bus arbitrates the request. Our contributions are listed as follows:

- We measure how much of the coherence traffic of the cache extension is necessary, and what the rest costs (Section IV).
- We propose an exact snoop filter, a structural mirror of the tag and valid bits of all DCUs that also covers lines being fetched, and give a cost model of the bus protocol that explains its benefit (Section V).
- We evaluate the filter in execution time, coherence correctness and FPGA resources, and measure directed invalidation as a possible next step (Sections VI and VII).

The rest of the paper is structured as follows. Section II discusses related work, Section III describes the cache extension under study, and Section IV analyzes its coherence traffic. The snoop filter is presented in Section V and evaluated in Section VI. Section VII reports on directed invalidation, and Section VIII concludes the paper.

II. RELATED WORK

Snoop filtering itself is not new. JETTY [2] places a small filter on the receiving side of the bus and uses it to skip snoops that would miss in a processor’s cache anyway, so that the energy of a tag lookup is not spent on them. Coarse-grain coherence tracking [3] and RegionScout [4] track the sharing state of large regions of memory instead, so that a region known to be unshared needs no broadcast at all.

In a bus-based soft multi-core system, the cost that matters is a different one. An unnecessary invalidation is expensive not because of the energy of the remote tag lookups but because of the serialization it causes. Only one invalidation is granted per clock cycle in the whole system, and the grant

is held back until every other cache is ready to accept it. For this reason we place the filter at the source of the request, where it can decide whether to arbitrate at all, and because the caches are small we can afford an exact copy of their tag arrays. The coherent RISC-V fabrics mentioned above are complete memory systems, whereas the design studied here is an extension attached to an unmodified core through its existing memory port, which is what confines its coherence traffic to a single shared bus.

III. THE CACHE EXTENSION UNDER STUDY

In this section we describe the cache extension of [1] as far as is needed to follow the measurements, keeping to the terminology of the original paper.

A. Multi-core RISC-V Architecture

The target system, shown in Fig. 1, consists of a scalable number of RISC-V Processing Elements (PEs). Each PE hosts an in-order RV32 core, a private Data Cache Unit (DCU), a scratchpad Instruction Tightly Coupled Memory (ITCM), and an Instruction-Bridge and a Data-Bridge that connect the core to the local memories and to a shared AXI4 crossbar, behind which sit a shared instruction memory, used as boot memory, and a coherent shared data memory. A single snoopy bus is connected to the DCU of every PE and manages the invalidation of cache lines between the private DCUs.

B. Data Cache Unit

The DCU is a two-way set-associative, write-through, no-allocate data cache with a Least Recently Used (LRU) replacement policy. Every write is forwarded to the shared data memory, and a write that misses does not install a line. Requests are processed in two sequential pipeline stages. The first stage accepts a request either from the local core, which can be a read request (READ REQ) or a write request (WRITE REQ), or from the snoopy bus, which can only be an invalidation request (INV REQ). For a core request, the first stage reads the valid bit from the Status-RAM and the tag from the Tag-RAM, and at the same time it forwards the request to the snoopy bus (a WRITE REQ is forwarded as an INV REQ). In the second stage the Hit Check Logic (HCL) decides between hit and miss and the Cache Control Logic (CCL) acts on the request. A READ REQ that misses issues a memory read request (MEM READ REQ) and fills the LRU way of the set with the returned line; a WRITE REQ is forwarded as a memory write request (MEM WRITE REQ); an INV REQ clears the valid bit of the addressed line if the line is resident. An INV REQ has priority over the local core. If it arrives while a core request occupies the first stage, that request is moved into a holding register and presented again once the INV REQ has moved on. A Stall Control Logic (SCL) stalls the first stage until the bus grants its forwarded request, and the second stage until the response (MEM RESP) of a MEM READ REQ or the grant of a MEM WRITE REQ arrives, that is, for the access latency of the shared data memory.

C. Snoopy Bus

The snoopy bus consists of two Round Robin (RR) arbiters, an Invalidation Table and a Link Register. The first arbiter, which we call the INV REQ arbiter from here on, arbitrates the INV REQs of all PEs and grants at most one per clock cycle. When an INV REQ is granted, its address is broadcast to the DCUs of all other PEs, which clear the addressed line if they hold it. The grant is withheld until every other DCU is able to accept the broadcast, so that no invalidation can be lost. The second arbiter handles the atomic Load-Reserved (LR) and Store-Conditional (SC) requests together with the Link Register. The bus also guarantees a Multiple-Readers-Single-Writer (MRSW) invariant. Plain READ REQs are not arbitrated, but while a core has an INV REQ pending for a line, which is recorded in the Invalidation Table, a READ REQ of another core to the same line is withheld until the INV REQ has been granted.

D. Reproduction and Experimental Setup

We implemented this architecture in SystemVerilog in the configuration given in the original paper: four PEs, each with an unmodified CV32E40P core and a private 2 KiB two-way DCU with 16-byte lines and 64 sets, a 256 KiB shared data memory and a 32 KiB shared instruction memory behind an AXI4 crossbar. A non-coherent version, in which each core accesses the shared data memory directly through its Data-Bridge and has no DCU, is built from the same sources by changing one parameter; it serves as the baseline for all comparisons. The shared memories are modelled with a configurable access latency, which we call the memory latency from here on. It is the number of clock cycles between the acceptance of a MEM READ REQ and the first data beat of its MEM RESP. Two cycles corresponds to on-chip block RAM (BRAM), as in the original design; eight and twenty cycles stand for off-chip memories.

Execution time is read from the cycle counter of the cores as the number of clock cycles until the slowest PE has finished. The benchmark kernels are the memory copy, matrix multiplication, 2-D convolution and Fast Fourier Transform (FFT) kernels of the original evaluation, which we refer to as memcopy, matmul, conv2d and FFT. Each of them partitions its data between the four PEs, and every run is checked against a golden result. In addition, each DCU carries hardware counters for the cycles in which it holds a core request (busy cycles), the cycles its first stage waits for a grant of the snoopy bus (snoop-stall cycles), the INV REQs it receives, and those among them that find a valid matching line to clear.

IV. ANALYSIS OF THE COHERENCE TRAFFIC

A. Time Spent Waiting for the Snoopy Bus

We express the snoop-stall cycles of a DCU as a fraction of its busy cycles. We call this the cache busy time of the core: it is the time the core spends with an access in its DCU, and not its total execution time, which also includes everything the core does without touching the cache. At a memory latency of two cycles this share is 14.3 % on memcopy,

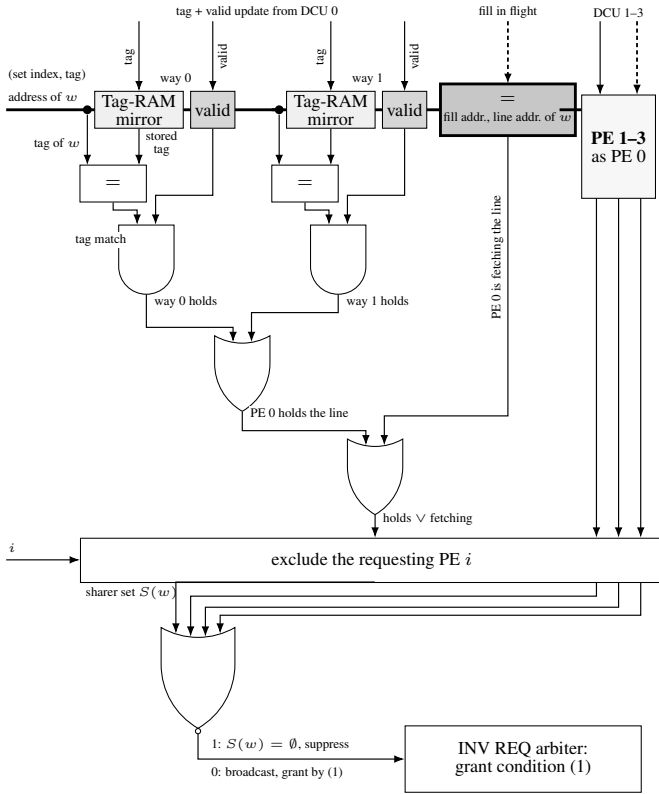


Fig. 2. Decision path of the snoop filter for the INV REQ selected by the arbiter, issued by PE i with address of w . The address bus reads every Tag-RAM mirror and valid bit at its set index. PE 1–3 are identical to PE 0. The fill comparator is shaded; it is the term of Section V-B.

together with its address, and the filter answers over the union of the mirror and these fill-in-flight signals. The fill signal stays asserted from the cycle the MEM READ REQ is issued until the cycle the line is written into the cache, which is also the cycle in which the mirror learns the new tag, so the line is covered at every moment by one of the two. This is not just a precaution. Over ten randomized interleavings of our coherence testbench, 20 out of 2,241 granted INV REQs addressed a line that no other DCU held but one DCU was fetching, and a mirror of resident lines alone would have suppressed each of them wrongly.

C. Correctness of Suppression

When the filter reports that no other DCU holds the addressed line or is fetching it, a broadcast of the INV REQ could not clear a line anywhere, so leaving it out cannot be observed. Everything else in the bus stays as it was: INV REQs that are needed are still arbitrated, broadcast and acknowledged as before, and the Invalidation Table still withholds READ REQs according to the MRSW invariant.

D. Cost Model

In this section we work out from the bus protocol what suppressing a broadcast is worth, and then check the result against the measurements. Let P be the number of PEs,

indexed by i and j . For a WRITE REQ w issued by PE i , let $a(w)$ be the clock cycle in which w enters the first stage of its DCU and $g(w)$ the cycle in which the corresponding INV REQ is granted. Let $\text{arb}(t)$ be the PE selected by the INV REQ arbiter in cycle t , and let $r_j(t) = 1$ if the DCU of PE j can accept an INV REQ in cycle t . The sharer set $S(w)$ is the set of PEs other than i whose DCU holds or is fetching the line addressed by w . The original protocol grants an INV REQ in the first cycle in which the arbiter selects it and every other DCU is ready to accept it:

$$g(w) = \min \left\{ t \geq a(w) : \text{arb}(t) = i \wedge \bigwedge_{j \neq i} r_j(t) \right\}. \quad (1)$$

A DCU is ready only while its first stage does not already hold an INV REQ, and an INV REQ leaves the first stage only when the second stage is free. The second stage is busy for the full memory latency whenever a MEM READ REQ or MEM WRITE REQ is in progress. On grant, one INV REQ is inserted into the first stage of each of the other $P-1$ DCUs, where it takes priority over the local core, so each of those pipelines loses one cycle if it had a core request to process. The cost of w to the whole system is therefore

$$C(w) = (g(w) - a(w)) + \sum_{j \neq i} \rho_j(g(w)), \quad (2)$$

where $\rho_j(t) \in \{0, 1\}$ is one if the DCU of PE j had a core request to delay in cycle t . The conjunction in (1) ties the pipelines of all PEs together: a WRITE REQ on PE i has to wait until a MEM READ REQ on PE j has completed, whether or not the two PEs share any data at all. This is the reason why the snoop-stall share of Section IV grows with the memory latency. And neither term of (2) depends on which lines the caches hold.

With the filter, the cost does depend on the cache state. When $S(w)$ is read from the mirror, the grant condition becomes $\text{arb}(t) = i \wedge (S(w) = \emptyset \vee \bigwedge_{j \neq i} r_j(t))$, and INV REQs are inserted into the other DCUs only when $S(w) \neq \emptyset$. For a WRITE REQ with an empty sharer set, both terms of (2) vanish and only the arbitration wait is left, which is non-zero only when two PEs present a WRITE REQ in the same cycle. If every WRITE REQ of a kernel has an empty sharer set, which is what Section IV found for all four kernels, the saving over the whole kernel is the entire coherence overhead less the arbitration collisions. That is the same saving an oracle would obtain by suppressing every broadcast.

Both terms of (2) can be observed, so the model can be checked rather than fitted. The snoop-stall cycles removed by the filter are the first term and give a lower bound on the saving; the same plus one cycle per broadcast received gives an upper bound. Per DCU, at a memory latency of two, the measured saving falls inside this bracket on every kernel: $\text{matmul } 120,539 \leq 243,100 \leq 326,373$ cycles, $\text{conv2d } 5,185 \leq 13,795 \leq 39,980$, $\text{FFT } 892 \leq 4,865 \leq 16,312$, $\text{memcpy } 1,792 \leq 2,302 \leq 7,946$.

In hardware, the mirror is one memory array per (PE, way) pair, addressed by the set index. The tags are kept outside any

TABLE I
EXECUTION TIME IN CLOCK CYCLES OF THE SLOWEST PE, FOR THE
VALUES PLOTTED IN FIG. 3. THE LAST TWO COLUMNS GIVE THE
REDUCTION RELATIVE TO THE NON-COHERENT SYSTEM WITHOUT A DATA
CACHE, FOR THE CACHE EXTENSION ALONE AND WITH THE PROPOSED
FILTER ADDED.

Kernel	Size	Execution cycles			Reduction	
		No cache	Cache	+Filter	Cache	+Filter
matmul	32 ²	277,621	171,142	140,223	38.4%	49.5%
	64 ²	2,191,981	1,337,850	1,094,750	39.0%	50.1%
	128 ²	16,899,384	9,461,615	8,118,893	44.0%	52.0%
conv2d	32 ²	73,268	38,120	35,526	48.0%	51.5%
	64 ²	311,454	168,177	154,382	46.0%	50.4%
	128 ²	1,277,966	683,079	625,110	46.5%	51.1%
FFT	256	113,092	85,093	80,228	24.8%	29.1%
	512	245,292	244,338	225,852	0.4%	7.9%
	1024	550,489	574,366	541,289	-4.3%	1.7%
FFT, blocked	256	121,213	93,087	88,604	23.2%	26.9%
	512	269,224	238,776	219,017	11.3%	18.6%
	1024	584,916	523,833	478,210	10.4%	18.2%

reset, because a tag is only compared when its valid bit is set, and this lets them map to distributed RAM (LUTRAM); the valid bits are kept in flip-flops, where they can be reset and read for all arrays at once. Our first implementation held both under an asynchronous reset, and Vivado mapped it entirely to flip-flops: 11,776 of them and 3,202 LUTs, against 512 and 408 here.

VI. EVALUATION AND PERFORMANCE ANALYSIS

A. Execution Time

Fig. 3 shows the execution time of the matmul, conv2d and FFT kernels for the problem sizes used in the original evaluation, and Table I lists the cycle counts behind it. For matmul and conv2d the cache extension reduces the execution time by roughly 40 % at every problem size, as in the original evaluation. The reduction does not fall off with the problem size because what has to stay resident is not the whole data set but only the reuse footprint of the kernel, the bytes a core comes back to before moving on. For matmul this is one row of the second matrix, $4N$ bytes for an $N \times N$ matrix of 32-bit words; for conv2d it is the three-row sliding window of the input, $12N$ bytes. At the largest problem size these come to 512 B and 1536 B, and both fit into the 2 KiB DCU.

For FFT this no longer holds. The original evaluation reports a uniform reduction of about 23 %. We match it at $N=256$ points, but the reduction disappears at $N=512$ and becomes negative at $N=1024$, where the cache makes the kernel slower than having no cache. The reuse footprint explains this too. The FFT touches all N complex points in every stage, $8N$ bytes in total, which is 2 KiB at $N=256$ and 8 KiB at $N=1024$. At $N=512$, doubling the associativity at 2 KiB recovers only 3.8 % of the execution time, whereas quadrupling the capacity recovers 24.5 %, so the cause is capacity. With an 8 KiB DCU, FFT reproduces the original result at every size (24.8, 24.8 and 24.5 %). With the 2 KiB stated in the original text it does not, so the reported result

implies either a larger cache or a smaller working set than described.

The collapse comes from the order in which the kernel accesses its data, and the fourth panel of Fig. 3 shows that it can be removed in software. A radix-2 butterfly in stage s pairs elements 2^s positions apart; once 2^s is a multiple of a block size B , the remaining stages fall apart into B independent transforms over strided columns. Running the early stages block by block and the late stages column by column keeps the working set inside the DCU, and since every butterfly still sees the same operands the output is bit-identical. At $N=1024$ the reduction obtained with the cache extension improves from -4.3 % to +10.4 %, and to 18.2 % with the filter; at $N=256$ the extra loop structure costs about 6 %. The filter removes coherence traffic, not cache misses, so its contribution stays at seven to eight percentage points in both FFT panels.

Compared with the coherent system without the filter, the filter improves the execution time by $1.222\times$ on matmul, $1.123\times$ on memcpy, $1.089\times$ on conv2d and $1.061\times$ on FFT, a geometric mean of $1.122\times$. We also compare against an oracle, that is, a bus that suppresses every INV REQ broadcast unconditionally. Simulating such a bus is only sound because none of the broadcasts are needed on these kernels. The filter reaches the oracle on every kernel, cycle for cycle, as the cost model of Section V-D predicts. The snoop-stall counter confirms the mechanism. On conv2d it falls from 20,848 cycles to 109, and on memcpy from 7,168 to 1. On FFT it falls only from 6,521 to 2,955, because the LR and SC requests used for synchronization still have to pass through the arbiters.

B. Memory Copy Latency

The memcpy kernel gave the most unexpected result. At a memory latency of two cycles, the coherent system with the cache extension is actually *slower* than the non-coherent system without any cache, at $0.975\times$ the speed of the baseline; with the filter it is $1.095\times$ faster. The slowdown is the cost of announcing private writes. Fig. 4 shows that this holds for every transfer size. From 4 to 32 KiB the cache extension is between 2.5 % and 7.6 % slower than the baseline, and the filter turns this into a gain of 8.2 % to 9.0 %. Raising the memory latency brings back the result of the original evaluation, with a latency reduction of 34 % at a memory latency of eight and 56 % at twenty, against the roughly 50 % reported there. A streaming kernel reuses no data, so when there is little memory latency to hide, only the coherence overhead is left.

C. Coherence Verification

The four benchmark kernels never exercise the case in which the filter has to let a broadcast through, so they cannot show that it is safe. Our coherence testbench does. It runs a producer/consumer exchange and a randomized four-core stress test over a small shared window, so that half of its INV REQs are useful, and with the filter it passes 3,112 checks. Among them is a version-monotonicity check: a core that has observed version k of a location must never afterwards observe

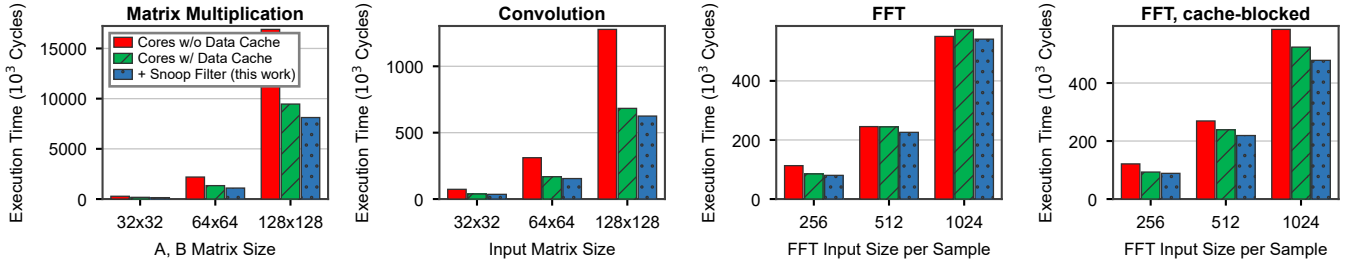


Fig. 3. Execution time of the benchmark kernels over the non-coherent system, the coherent system with the cache extension, and the coherent system with the proposed snoop filter. The fourth panel is the FFT with its stages reordered so that its working set fits into the cache.

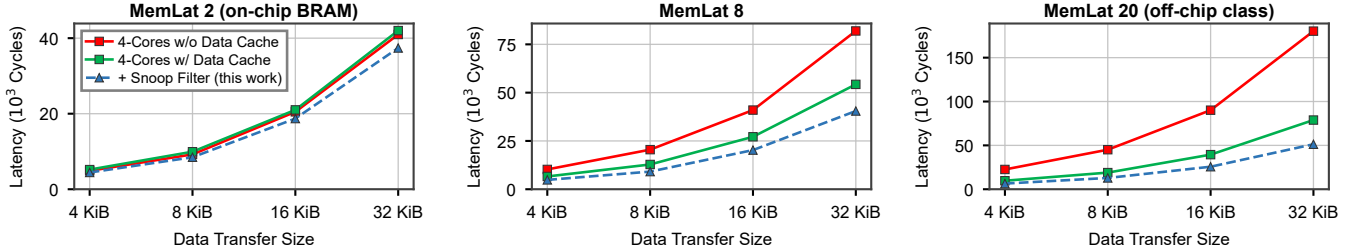


Fig. 4. Memory copy latency for several data transfer sizes at memory latencies of two, eight and twenty cycles. At two the cache extension is slower than the non-coherent system at every size; the $\approx 50\%$ reduction of the original evaluation appears between eight and twenty.

TABLE II

POST-PLACE-AND-ROUTE RESOURCE UTILIZATION OF THE COHERENT MULTI-CORE SYSTEM WITH THE PROPOSED SNOOP FILTER, AMD/XILINX XCZU7EV AT 60 MHZ.

Module	LUTs	FFs	BRAMs	DSPs
Processing element ($\times 4$)	17,833	9,948	32	20
RISC-V core, CV32E40P	16,929	8,404	0	20
ITCM + Instr.-Bridge	252	36	32	0
Data-Bridge	319	32	0	0
AXI4 masters ($\times 3$ per PE)	333	1,476	0	0
Cache extension	6,540	4,628	20	0
Data Cache Unit ($\times 4$)	5,816	3,998	20	0
Snoopy bus	728	630	0	0
snoop filter (this work)	408	512	0	0
AXI4 crossbar	2,125	279	0	0
Shared data memory, 256 KiB	724	150	64	0
Shared instr. memory, 32 KiB	506	86	8	0
Whole device, as reported by the tool	27,845	15,670	124	20
Available	230,400	460,800	312	1,728
Utilisation (%)	12.09	3.40	39.74	1.16

an older version, which is the symptom of a lost invalidation. The filter suppressed 282 of 921 broadcasts and delivered every one that was needed.

D. Resource Utilization and Timing

Table II gives the post-place-and-route resource utilization with the filter, obtained with Vivado 2025.1 for an AMD/Xilinx Zynq UltraScale+ XCZU7EV at 60 MHz, which is the clock frequency of the original design. The filter costs 1,064 look-up tables (LUTs) and 516 flip-flops on top of the coherent system without it, under 4 % of that system, and no BRAM or DSP blocks. Both systems meet all timing

constraints, with a worst negative slack of +1.199 ns with the filter and +1.265 ns without, so the filter costs 66 ps on a 16.667 ns clock period. Both numbers come from the same Vivado session, because separate runs of this design vary by up to 1.7 ns in slack. At 75 MHz an earlier pair of builds had only the unfiltered system meeting timing, on a path that is 86 % routing delay, so we evaluate at 60 MHz.

VII. DIRECTED INVALIDATION

Since the mirror knows not only *whether* another DCU holds the line but also *which* ones do, the bus could deliver the INV REQ only to the members of $S(w)$ and wait only for those. We tried this on a stencil kernel in which each PE updates a band of rows and exchanges the boundary rows with its neighbours. It delivers 168 INV REQs, all of them useful, but the execution time improves by only 0.1 % to 0.3 %. Once suppression has removed every WRITE REQ with an empty sharer set, whatever cost is left in (2) belongs to writes that some other DCU has to be told about. Even on this kernel, which does share data, suppression alone is worth $1.11\times$ at a memory latency of twenty.

VIII. CONCLUSION

On the benchmark kernels of its own evaluation, the seamless cache extension we reproduced spends 6 % to 33 % of each core's cache busy time on invalidation requests that invalidate nothing. An exact mirror of the tag and valid bits of all DCUs, covering the lines being fetched as well, lets the bus suppress this traffic for 1,064 LUTs and is worth $1.122\times$.

REFERENCES

- [1] A. Kamaleldin, M. Nickel, S. Wu and D. Göhringer, “Seamless Cache Extension for FPGA-based Multi-Core RISC-V SoC,” in *Proc. IEEE 37th Int. System-on-Chip Conf. (SOCC)*, 2024.
- [2] A. Moshovos, G. Memik, B. Falsafi and A. Choudhary, “JETTY: Filtering Snoops for Reduced Energy Consumption in SMP Servers,” in *Proc. 7th Int. Symp. High-Performance Computer Architecture (HPCA)*, 2001.
- [3] J. F. Cantin, M. H. Lipasti and J. E. Smith, “Improving Multiprocessor Performance with Coarse-Grain Coherence Tracking,” in *Proc. 32nd Int. Symp. Computer Architecture (ISCA)*, 2005, pp. 246–257.
- [4] A. Moshovos, “RegionScout: Exploiting Coarse Grain Sharing in Snoop-Based Coherence,” in *Proc. 32nd Int. Symp. Computer Architecture (ISCA)*, 2005, pp. 234–245.
- [5] W. Terpstra, “TileLink: A Free and Open-Source, High-Performance Scalable Cache-Coherent Fabric Designed for RISC-V,” in *RISC-V Workshop*, 2017.
- [6] J. Balkind *et al.*, “OpenPiton+Ariane: The First Open-Source, SMP Linux-booting RISC-V System Scaling from One to Many Cores,” in *Workshop on Computer Architecture Research with RISC-V (CARRV)*, 2019.
- [7] A. Li *et al.*, “CIFER: A Cache-Coherent 12nm 16mm² SoC with Four 64-Bit RISC-V Application Cores, 18 32-Bit RISC-V Compute Cores, and a 1541 LUT6/mm² Synthesizable eFPGA,” in *IEEE Int. Solid-State Circuits Conf. (ISSCC)*, 2023.